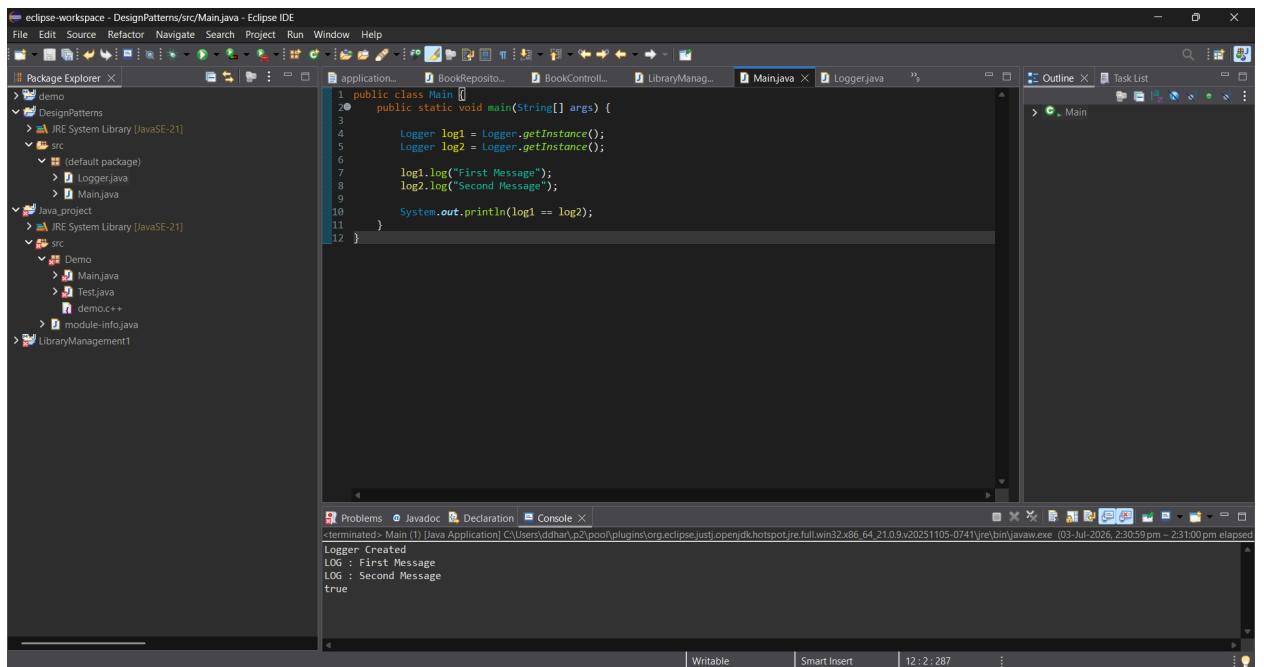
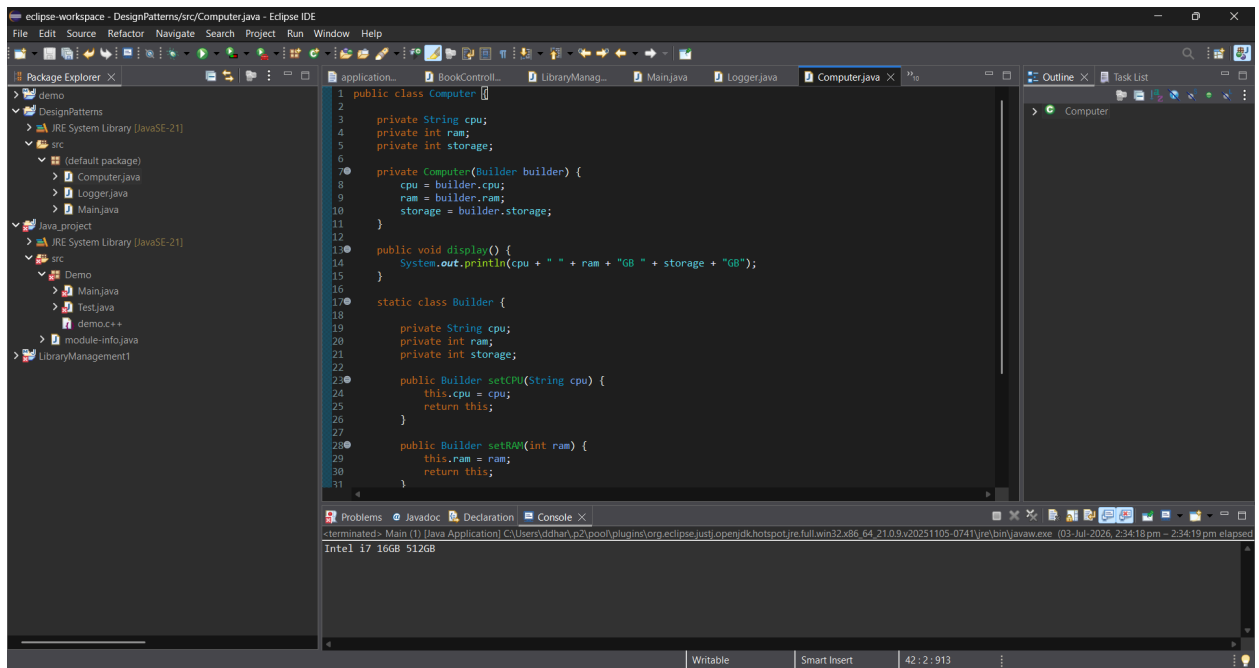


1. Logger.java

```
public class Logger {  
  
    private static Logger instance;  
  
    private Logger() {  
        System.out.println("Logger Created");  
    }  
  
    public static Logger getInstance() {  
        if(instance == null) {  
            instance = new Logger();  
        }  
        return instance;  
    }  
  
    public void log(String message) {  
        System.out.println("LOG : " + message);  
    }  
}
```



3.



```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Computer pc = new Computer.Builder()
            .setCPU("Intel i7")
            .setRAM(16)
            .setStorage(512)
            .build();
```

```
        pc.display();
```

```
    }
}
```

4. public interface PaymentProcessor {

```
    void processPayment();
```

```
}
```

```
public class PayPalGateway {
```

```
    public void makePayment() {
```

```
        System.out.println("Payment through PayPal");
```

```

    }
}
public class PayPalAdapter implements PaymentProcessor {

    private PayPalGateway gateway = new PayPalGateway();

    public void processPayment() {
        gateway.makePayment();
    }
}

```

5. Notifier.java

```

public interface Notifier {
    void send();
}

```

EmailNotifier.java

```

public class EmailNotifier implements Notifier {

    public void send() {
        System.out.println("Email Sent");
    }
}

```

NotifierDecorator.java

```

public abstract class NotifierDecorator implements Notifier {

    protected Notifier notifier;

    public NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }
}

```

SMSNotifierDecorator.java

```

public class SMSNotifierDecorator extends NotifierDecorator {

```

```

    public SMSNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    public void send() {
        notifier.send();
        System.out.println("SMS Sent");
    }
}

```

Main.java

```

public class Main {

    public static void main(String[] args) {

        Notifier notifier = new SMSNotifierDecorator(new EmailNotifier());

        notifier.send();
    }
}

```

6. Image.java

```

public interface Image {
    void display();
}

```

RealImage.java

```

public class RealImage implements Image {

    private String file;

    public RealImage(String file) {
        this.file = file;
        System.out.println("Loading " + file);
    }

    public void display() {

```

```
        System.out.println("Displaying " + file);
    }
}
```

ProxyImage.java

```
public class ProxyImage implements Image {

    private ReallImage reallImage;
    private String file;

    public ProxyImage(String file) {
        this.file = file;
    }

    public void display() {

        if(reallImage == null)
            reallImage = new ReallImage(file);

        reallImage.display();
    }
}
```

Main.java

```
public class Main {

    public static void main(String[] args) {

        Image img = new ProxyImage("sample.jpg");

        img.display();
        img.display();
    }
}
```

Exercise 7: Observer Pattern

```

interface Observer{
    void update(int price);
}

class MobileApp implements Observer{
    public void update(int price){
        System.out.println("Mobile Price : "+price);
    }
}

class WebApp implements Observer{
    public void update(int price){
        System.out.println("Web Price : "+price);
    }
}

class StockMarket{

    java.util.ArrayList<Observer> list=new java.util.ArrayList<>();

    void register(Observer o){
        list.add(o);
    }

    void notifyObservers(int price){
        for(Observer o:list)
            o.update(price);
    }
}

public class Main{

    public static void main(String args[]){

        StockMarket stock=new StockMarket();

        stock.register(new MobileApp());
    }
}

```

```
        stock.register(new WebApp());

        stock.notifyObservers(250);
    }
}
```

Exercise 8: Strategy Pattern

```
interface PaymentStrategy {
    void pay(int amount);
}
```

```
class CreditCardPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Credit Card Payment "+amount);
    }
}
```

```
class PayPalPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("PayPal Payment "+amount);
    }
}
```

```
class PaymentContext {

    PaymentStrategy strategy;

    PaymentContext(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    void pay(int amount) {
        strategy.pay(amount);
    }
}
```

```
public class Main{

    public static void main(String args[]){

        PaymentContext payment=new PaymentContext(new CreditCardPayment());

        payment.pay(5000);

        payment=new PaymentContext(new PayPalPayment());

        payment.pay(3000);
    }
}
```

Exercise 9: Command Pattern

```
interface Command{
    void execute();
}

class Light{

    void on(){
        System.out.println("Light ON");
    }

    void off(){
        System.out.println("Light OFF");
    }
}

class LightOnCommand implements Command{

    Light light;
```



```

    LightOnCommand(Light light){
        this.light=light;
    }

    public void execute(){
        light.on();
    }
}

class LightOffCommand implements Command{

    Light light;

    LightOffCommand(Light light){
        this.light=light;
    }

    public void execute(){
        light.off();
    }
}

class RemoteControl{

    Command command;

    void setCommand(Command command){
        this.command=command;
    }

    void pressButton(){
        command.execute();
    }
}

public class Main{

```

```
public static void main(String args[]){

    Light light=new Light();

    RemoteControl remote=new RemoteControl();

    remote.setCommand(new LightOnCommand(light));
    remote.pressButton();

    remote.setCommand(new LightOffCommand(light));
    remote.pressButton();
}
}
```

Exercise 10: MVC Pattern

```
class Student{

    String name;
    int id;
    String grade;

    Student(String name,int id,String grade){
        this.name=name;
        this.id=id;
        this.grade=grade;
    }
}

class StudentView{

    void display(Student s){
        System.out.println(s.name);
        System.out.println(s.id);
        System.out.println(s.grade);
    }
}
```

```
}
```

```
class StudentController{
```

```
    Student model;
```

```
    StudentView view;
```

```
    StudentController(Student model,StudentView view){
```

```
        this.model=model;
```

```
        this.view=view;
```

```
    }
```

```
    void updateName(String name){
```

```
        model.name=name;
```

```
    }
```

```
    void show(){
```

```
        view.display(model);
```

```
    }
```

```
}
```

```
public class Main{
```

```
    public static void main(String args[]){
```

```
        Student s=new Student("Nivashini",101,"A");
```

```
        StudentView view=new StudentView();
```

```
        StudentController controller=new StudentController(s,view);
```

```
        controller.show();
```

```
        controller.updateName("Priya");
```

```
        controller.show();
```

```
    }
```

```
}
```

Exercise 11: Dependency Injection

```
interface CustomerRepository{
```

```
    String findCustomerById(int id);  
}
```

```
class CustomerRepositoryImpl implements CustomerRepository{
```

```
    public String findCustomerById(int id){  
        return "Customer ID : "+id;  
    }  
}
```

```
class CustomerService{
```

```
    CustomerRepository repository;
```

```
    CustomerService(CustomerRepository repository){  
        this.repository=repository;  
    }
```

```
    void findCustomer(int id){  
        System.out.println(repository.findCustomerById(id));  
    }  
}
```

```
public class Main{
```

```
    public static void main(String args[]){
```

```
        CustomerRepository repository=new CustomerRepositoryImpl();
```

```
        CustomerService service=new CustomerService(repository);
```

```
    service.findCustomer(101);  
  }  
}
```